

COB295 Team Project - iBay

Group 06 Members:

- Shravan
- Ashley
- Max
- Qayyis

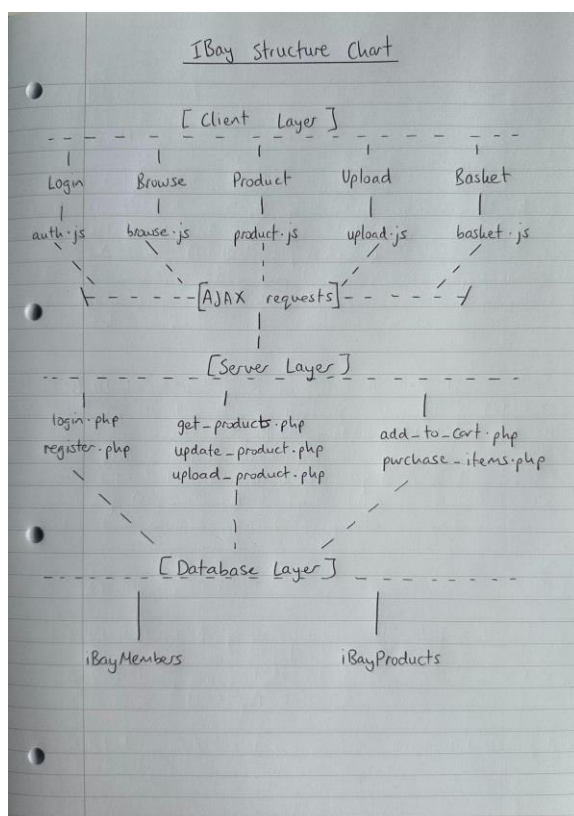
Member	Contribution
Shravan	Login, Authentication, Basket, backend PHP
Ashley	Browse and filter page, backend filter logic (PHP)
Max	Product Page, add to basket logic, seller rating logic, backend PHP
Qayyis	Upload page, add/edit/view products logic, backend PHP

URL: <http://group06.sci-project.lboro.ac.uk/iBay/frontend/html/main-G06.html>

Email: test@gmail.com

Password: 12345678

Introduction



This project involves the design and development of *iBay*, a full-stack client server web application that simulates an online auction and e-commerce platform. The system allows users to register, authenticate, browse items, list products for sale, and complete purchases through an interactive and responsive interface.

The application is built using a combination of modern web technologies, including HTML and CSS for structure and presentation, JavaScript and jQuery for client-side interactivity, and PHP with a MySQL database for server-side processing and persistent data storage. Communication between the client and server is handled by asynchronous requests (AJAX) using JSON, enabling a smooth and dynamic user experience without unnecessary page reloads.

The system follows a clear client server architecture. The client side is responsible for handling user interactions, performing initial validation, and dynamically updating the interface, while the server side manages authentication, data validation, business logic, and secure database operations. Key features of the system include user authentication with session management, item searching and filtering, product listing and management, and a basket system that integrates both client-side storage and server-side processing.

This report presents an overview of the system's functionality, design decisions, and implementation. It highlights how different technologies are integrated to create a cohesive application, discusses security considerations such as SQL injection prevention, and evaluates the strengths and limitations of the system. The report also reflects on the use of client-side and server-side techniques within the project and justifies the design choices made.

Figure 1 demonstrates the layered client-server architecture used within the *iBay* system. The client layer manages user interaction and AJAX communication, while the server layer processes business logic and interacts securely with the MySQL database.

Client-Side vs Server-Side Implementation

Web applications typically follow a client server architecture, where responsibilities are divided between the client (browser) and the server. This separation improves performance, scalability, and security when implemented effectively.

Client-Side Implementation

Client-side processing is executed in the user's browser using technologies such as HTML, CSS, JavaScript, and jQuery. Its primary role is to manage user interaction, update the interface dynamically, and perform initial input validation before data is sent to the server.

In the *iBay* system, client-side techniques are used to enhance responsiveness and usability. For example, during login and registration, JavaScript intercepts form submissions and sends asynchronous AJAX requests to the server. This prevents full page reloads and allows error messages (such as invalid credentials or missing fields) to be displayed instantly.

Similarly, the basket functionality is largely managed on the client side using localStorage. Items added to the basket are stored and rendered dynamically without requiring immediate server interaction. This reduces unnecessary server requests and provides a faster, smoother user experience.

Advantages of Client-Side Processing

- Faster response times as no server communication is required for simple tasks
- Improved user experience through dynamic updates (AJAX)
- Reduced server load
- Enables interactive features such as real-time validation

Disadvantages of Client-Side Processing

- Less secure, as code can be viewed and manipulated by users
- Cannot be trusted for critical validation or sensitive operations
- Browser compatibility issues may arise

Server-Side Implementation

Server-side processing is handled by backend technologies such as PHP and MySQL. It is responsible for managing application logic, enforcing security, and interacting with the database.

In the iBay system, all critical operations are handled on the server. For instance, during authentication, login.php validates user credentials against stored records in the MySQL database. Passwords are securely stored using hashing, and prepared statements are used to prevent SQL injection attacks.

The server is also responsible for processing purchases. When a user clicks “Buy Now”, the client sends product IDs to purchase_items.php, where the server validates the request and updates the database accordingly. This ensures data integrity and prevents users from manipulating transaction data.

Advantages of Server-Side Processing

- More secure, as logic and data are hidden from the user
- Centralised control over application behaviour
- Reliable database interaction and data validation
- Essential for sensitive operations such as authentication and transactions

Disadvantages of Server-Side Processing

- Slower response times due to network communication
- Increased server load with high user traffic
- Requires more resources and infrastructure

Integration in the iBay System

The iBay system demonstrates a balanced use of both client-side and server-side techniques.

- Client-side validation is used to quickly check user input (e.g. email format, password length), improving usability.
- Server-side validation is then performed to ensure correctness and security before interacting with the database.
- AJAX is used as a bridge between the two, allowing effortless communication without page reloads.
- The basket system uses client-side storage for speed, while final purchase processing is handled securely on the server.

This approach ensures that the system remains both efficient and secure.

Choosing Between Client-Side and Server-Side Approaches

When designing a web application, the choice between client-side and server-side processing depends on the nature of the task.

- Client-side solutions should be used for tasks that require fast interaction and do not involve sensitive data, such as UI updates or basic validation.
- Server-side solutions should be used for operations involving security, data integrity, or database access, such as authentication and transactions.

In practice, most modern applications including iBay use a combination of both approaches to achieve optimal performance, usability, and security.

Login and Registration Page



The login form features the iBay logo at the top. Below it is the heading "Sign in to your Account". There are two input fields: "Email address" and "Password". A blue "Sign In" button is positioned below the password field. At the bottom, there is a link: "Don't have an account? [Create an account](#)".

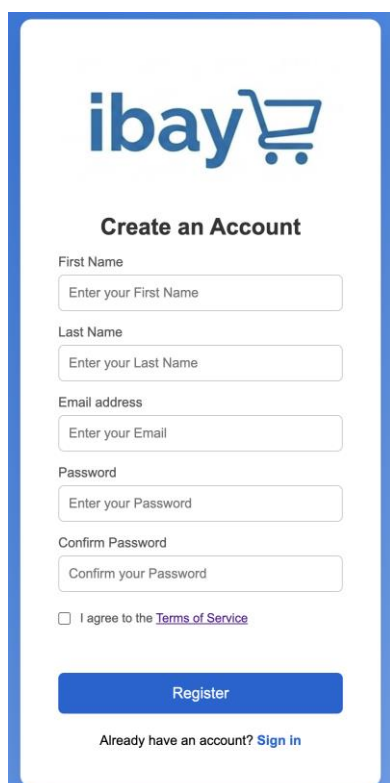
The login page provides a simple sign-in form with email and password fields. When the user submits, JavaScript prevents default form reload and sends an asynchronous POST request to backend/login.php. This gives a smoother UX because feedback appears instantly on the same page.

login.php performs server-side checks:

- Verifies request method is POST
- Validates that email/password are present
- Looks up the user in iBayMembers by email
- Verifies password securely with password_verify()

If authentication succeeds, the server stores user data in the PHP session (user_id, user_email) and returns JSON with a redirect target (browse.html?id=<userId>). If it fails, the API returns a clear error (Invalid email or password), which is displayed inline on the page.

A useful feature is session awareness: on page load, auth.js calls backend/me.php. If the user is already logged in, they are automatically redirected to browse rather than seeing login again.



The registration form features the iBay logo at the top. Below it is the heading "Create an Account". There are five input fields: "First Name", "Last Name", "Email address", "Password", and "Confirm Password". A checkbox labeled "I agree to the [Terms of Service](#)" is located below the password fields. A blue "Register" button is positioned below the confirm password field. At the bottom, there is a link: "Already have an account? [Sign in](#)".

The registration page is a two-step flow:

1. Basic details step

The user enters first name, last name, email, password, confirm password, and accepts Terms of Service.

2. Preference step

The user selects at least one preferred shopping genre (e.g., Fashion, Electronics, Books).

Selected genres are stored in a hidden field (preferred_genres) and submitted to the backend.

Client-side validation in auth.js checks:

- Valid email format
- Minimum password length (8+)
- Password confirmation match
- Terms checkbox selected
- At least one genre selected in step 2

After validation, an AJAX POST request is sent to backend/register.php.

register.php then performs authoritative server-side validation:

- Required-field checks
- Email format validation
- Password rule enforcement
- Duplicate email check
- Genre presence check

The password is hashed using `password_hash()` before storage, and data is inserted with prepared statements into `iBayMembers` (email, password, firstName, lastName, preferredGenres). On success, the user is redirected back to login with `?registered=1`, where a success message ("Account created. Please sign in.") is displayed.

Security & SQL Injection Prevention

Security is a critical aspect of any web application, particularly when handling user authentication and sensitive data. One of the most common vulnerabilities in web systems is SQL injection, where malicious users attempt to manipulate database queries by injecting harmful SQL code through input fields.

In the iBay system, SQL injection is mitigated using prepared statements. Instead of directly embedding user input into SQL queries, parameterised queries are used to separate SQL logic from user-provided data. For example:

```
// Look up user record by email using a prepared statement
$stmt = mysqli_prepare($conn, 'SELECT id, email, password FROM iBayMembers WHERE email = ?');
```

This ensures that user input (email) is treated strictly as data rather than executable SQL code. By binding parameters using:

```
mysqli_stmt_bind_param($stmt, 's', $email);
```

the system prevents malicious inputs (e.g. `' OR 1=1 --`) from altering the structure of the SQL query.

Additionally, passwords are not stored in plain text. Instead, they are securely hashed, and the `password_verify()` function is used to validate user credentials. This enhances security by ensuring that even if the database is compromised, user passwords remain protected.

Overall, by combining prepared statements with secure password hashing, the iBay system enforces strong server-side security. These measures ensure data integrity, protect user credentials, and align with best practices for modern web application development.

Session Management

Once the user is successfully authenticated, a session is created:

```
$_SESSION['user_id'] = (int) $user['id'];
```

This allows the system to maintain the user's logged-in state across different pages. The user is then redirected to the main browsing page of the application.

Why this design is strong

- Clear client-server separation: UI/UX checks on client; security and data integrity on server.
- Good responsiveness: AJAX submission avoids full page reloads.
- Security-aware: prepared SQL statements + hashed passwords.
- Better onboarding UX: multi-step registration and preference capture for personalisation.
- Session handling: avoids redundant sign-in for authenticated users.

Known Limitations

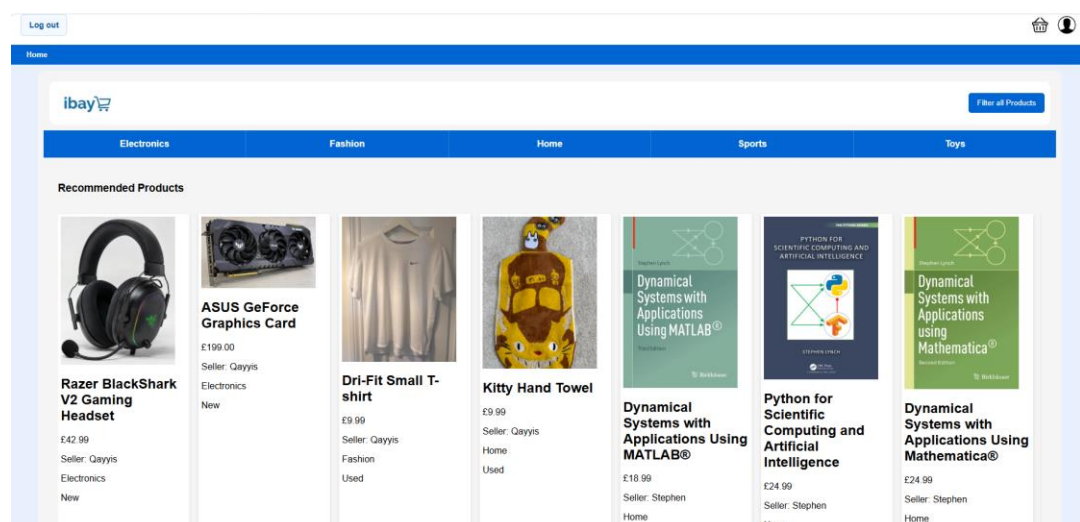
- The email validation on client side is basic (includes ("@")) and could be stricter.
- There is no explicit password strength meter (e.g., symbol/uppercase guidance).
- No rate-limiting or account lockout is shown for repeated failed logins.
- No "remember me" or password recovery functionality
- No password reveal functionality for user when entering password

Technology	Usage in iBay
HTML	Page Structure
CSS	Layout and Styling (blue theme)
JavaScript	Validation and UI Logic
jQuery	AJAX requests, DOM manipulation
AJAX	Login, product loading
PHP	Backend processing
MySQL	Storing user and product data
JSON	Send user and product credentials around the website

Browse Page

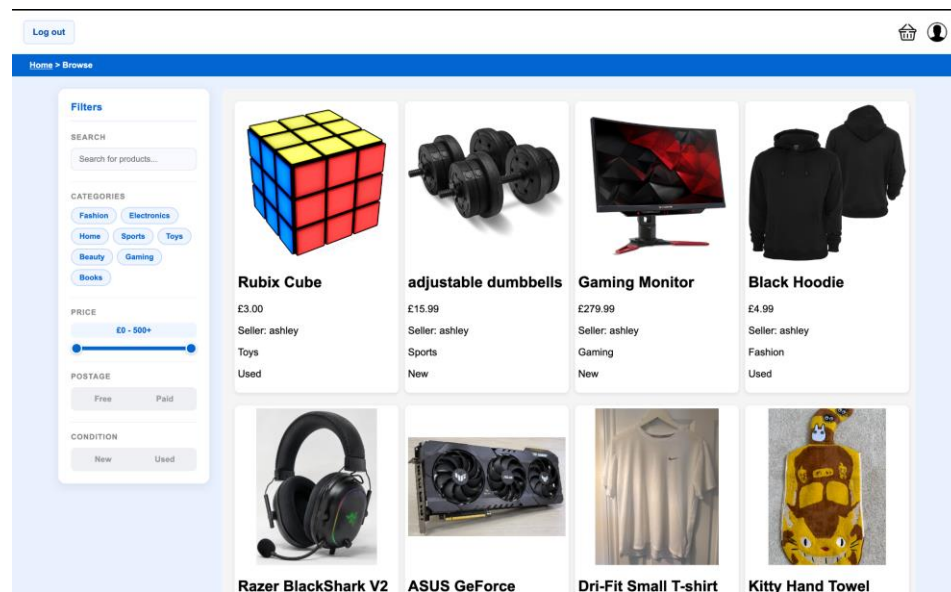
There are two pages for viewing products in the database, a main browse page(1) that acts as the home page as well as an advanced search page(2) to give the user the functionality to filter the products to their preferences.

(1)



Upon a successful login attempt the user is sent to the main browse page where they can navigate to the other pages of the system and/or view products on the carousels. On page load a jQuery AJAX GET request is sent using `get_recommended_products.php` and `get_latest_products.php` to retrieve the relevant products to then be displayed to the screen. `Get_recommended_products.php` queries the user table for the users preferred product categories that were requested when signing up, it then selects the corresponding products from the products table. `Get_latest_products` simply selects the products from the product table in descending order

(2)



The advanced search page gives the user the functionality to filter the products to their needs including category, price, postage, condition as well as a text search. Using an event listener in `browse_category.js` the rendered products can be updated as the user changes the filters without a page reload.

```

window.addEventListener("DOMContentLoaded", () => {
  updateHeaderLinks();
  setInitialFiltersFromUrl();
  setupPriceSliderEvents();
  setupFormEvents();
  fetchAndRenderProducts();
});

```

An async function `fetchAndRenderProducts` is used to allow asynchronous execution so the user can interact with the page while the functions task is completing.

When the user does change the value of a filter executing the render function a php file `filter.php(*)` builds an SQL statement using the various filters and returns all relevant products to be rendered to the screen.

(*)

```

try {
  const response = await fetch("../backend/filter.php", {
    method: "POST",
    body: buildFormData()
  });
}

```

`filter.php` works by starting with a base SQL statement then uses each filter to apply the WHERE conditions (Base SQL)

```

// Base SQL includes a JOIN to get the seller's name from the iBayMembers table
$sql = "SELECT p.*, m.firstName AS sellerName
FROM iBayProducts p
JOIN iBayMembers m ON p.sellerId = m.id
WHERE 1=1";

```



```
// SEARCH BAR LOGIC
// select products where productName or category contains the search term
if (!empty(trim($searchTerm))) {
    $sql .= " AND (productName LIKE ? OR category LIKE ?)";
    $searchTermWildcard = "%" . $searchTerm . "%";
    $params[] = $searchTermWildcard;
    $params[] = $searchTermWildcard;
    $types .= "ss";
}
```

(e.g. Add search bar filter)

After repeating for each filter, the full SQL statement is built and returns the required products to the js function to output to the user.

Product Page

The product page displays detailed information about a single item when a user chooses an item to view, whether that is from browse or the search page. When the user clicks on a listing, the product ID is passed through the URL (e.g. product.html?id=2). Then jQuery retrieves and displays the full item details.

Page Load and Data Retrieval

On loading the page, a jQuery AJAX GET request is sent to the backend using get_product.php, using the product ID from the URL. The PHP script queries iBayProducts using MySQL and performs a LEFT JOIN with iBayMembers to retrieve both product details and seller information in a single database call. The response is then returned as JSON and injected into the relevant HTML elements by JavaScript.

The following fields are displayed on the page from the database:

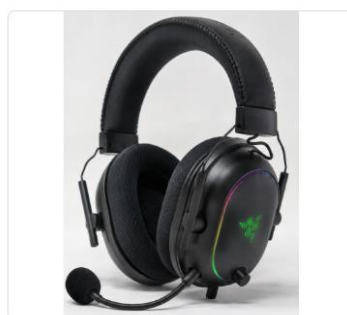
- productName - title of the listing
- Price and postage - displayed in the price box
- Item_condition - shown beneath the product title
- Description - shown in the item description section
- Image_url_1 and image_url_2 - loaded as the main image and thumbnails
- Seller_email and seller_since - shown in the seller details table
- rating_total and rating_count - used to calculate the seller rating out of 5 stars

Image Thumbnail Swap

A client-side JavaScript function changelImage() allows the user to click on a thumbnail image to update the main product image displayed on the page. This is handled entirely in the browser with no server interaction, showing client-side DOM manipulation.

Razer BlackShark V2 Gaming Headset

Condition: new



£42.99

Postage: £3.49

Seller: Qayyis786@gmail.com

Add to Basket

Add to Basket

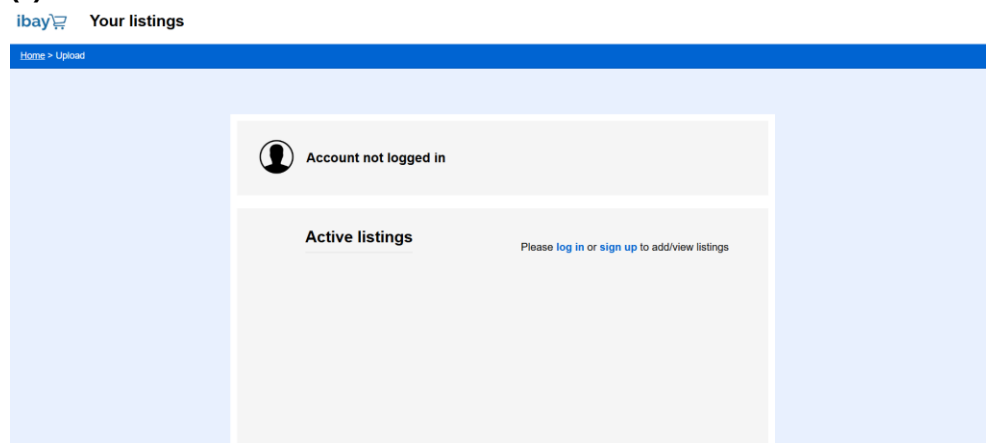
When the user clicks the Add to Basket button, a jQuery AJAX POST request is sent to the backend using `add_to_cart.php`. The PHP script checks that the user is logged in via the session, confirms the existence of the product with a matching ID in `iBayProducts`, and then appends the product ID to the basket column in the user's `IBayMembers` record, stored as a comma-separated list. If successful, the user is redirected to `basket.html`; otherwise, if the user is not logged in, an error message is returned as JSON and displayed via a browser alert.

Upload Page

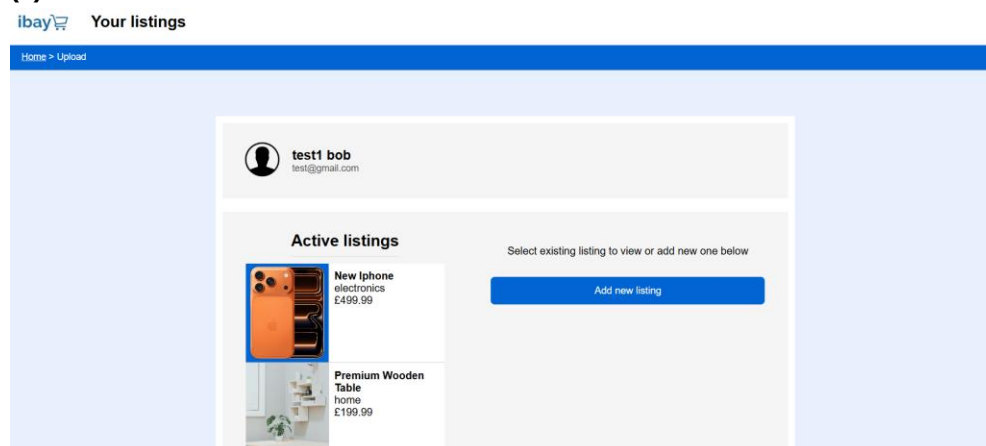
The upload page allows authenticated users to create new listings and view/edit existing ones associated with their account. The interface is designed around multiple UI states, allowing the content displayed on the page to change dynamically without undergoing page reloads. JavaScript controls these states through DOM manipulation, while AJAX is used to communicate asynchronously with the backend.

Initially, the user is presented with one of two default views depending on whether they are authenticated:

(1)



(2)



If the user is not logged in, the interface displays links directing them to the login or registration pages as seen in (1). If authenticated, the user is presented with their account details, active listings, and listing management controls (2).

The Upload page is comprised of three interface states:

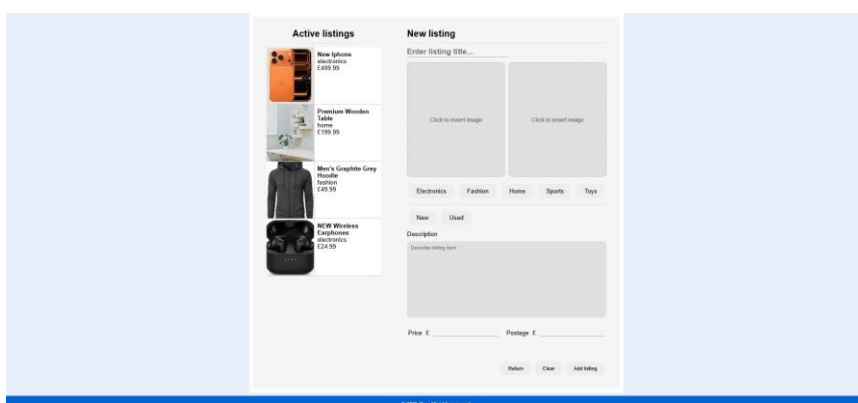
- **Empty State**
- **Add Listing State**
- **Existing Listing State**

The Empty State

This is the default state of the Upload page as shown above. This state displays the user's account information, including their name and email address, and their existing listings. The existing listings are loaded dynamically using an asynchronous AJAX request to my_products.php with the returned data displayed as a vertically scrollable carousel.

Selecting the 'add new listing' button transitions the interface into the Add Listing state, while selecting an existing product transitions the page into the Existing Listing state.

The 'Add Listing' State



The Add Listing State provides a form-based interface allowing users to create new product listings. The form collects the following information:

- Listing title
- 2 Product images
- Product category
- Product condition
- Description
- Price
- Postage cost

The interface utilises radio buttons for category and condition selection, while image uploads are handled through presented upload boxes. These upload boxes also incorporate live preview functionality through JavaScript. When the user selects an image, upload.js

updates the preview area using createObjectURL(), improving responsiveness and the overall UX:

```
const imageUrl = URL.createObjectURL(file);
box.style.backgroundImage = `url('${imageUrl}')`;
box.style.border = 'none';
box.style.backgroundColor = 'transparent';
```

Client-side validation is performed before submission to ensure all required fields are completed correctly. These include:

New listing

Men's Graphite Grey Hoodie



Click to insert image

Please upload the second image.

Electronics Fashion Home Sports Toys

Please choose a category.

New Used

Description

Describe listing item

Please enter a description.

Price £ 49.99 Postage £

Please enter valid postage.

Please fill in all fields.

Return Clear Add listing

- Empty field detection
- Positive numeric validation for price and postage
- Category and condition selection checks
- Required image upload validation

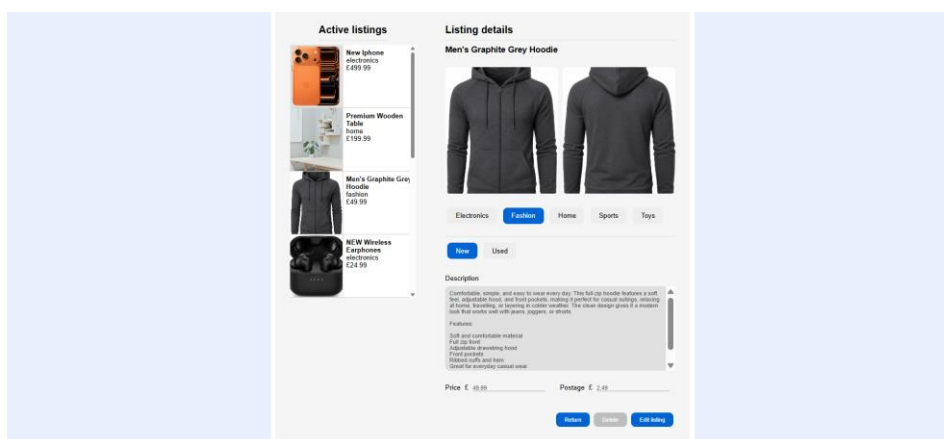
Validation errors are displayed individually beside the relevant fields, improving clarity and user experience.

Once validation succeeds, the form data is submitted asynchronously using AJAX to upload_product.php. The PHP script performs validation through:

- Verifying the user session
- Ensuring the request method is POST
- Validating uploaded image file types (JPEG, PNG, and WEBP)
- Enforcing image size restrictions (< 5MB)
- Using prepared SQL statements

Additionally, the user has the option to cancel adding a new listing and returning to the empty state by clicking the 'return' button. They can also reset all form fields (including selected images) with the 'clear' button.

'Existing Listing' State



The Existing Listing State allows users to view, delete and modify products they have previously uploaded. When a listing is selected from the provided carousel, that product's information is inserted into the interface through upload.js.

Initially, the listing information is displayed in a read-only format and does not allow the user to edit the listing. To do so, they first click the 'Edit Listing' button. Doing so, the display switches to edit mode, and allows the user to edit the fields using the same UI structure as if adding a new listing. This also enables the delete button, which allows the user to delete the listing. This is accomplished through upload.js calling delete_product.php.

If the user is editing an existing listing, they can click on the 'save' button to submit those changes. Once the user clicks submit, the JavaScript, upload.js, performs client-side validation like the Add Listing process. The javascript then sends the updated data asynchronously to update_product.php.

The backend validates the request, following a similar procedure to `upload_product.php`, and updates the database using prepared SQL statements. Individual updated image upload is performed, provided the corresponding image field was changed during the editing process.

Why this design is strong

- Clear separation between client-side interactivity and server-side security
- Dynamic UI states improve responsiveness without page reloads
- AJAX communication creates a smoother user experience
- Client-side validation improves usability and reduces unnecessary server requests
- Server-side validation ensures data integrity and security
- Prepared SQL statements help prevent SQL injection attacks
- Session validation ensures only authenticated users can manage listings

Known Limitations

- No drag-and-drop image upload functionality is implemented
- No automatic image compression or resizing is performed
- No manual option to crop image or resize to fit a suitable frame
- File validation is limited to type and size checks
- The system does not support draft listings or autosave functionality

Basket Page

The basket page (`basket.html`) consists of three main components:

- **Basket list** – displays all items currently added by the user
- **Order summary** – dynamically calculates total price, postage, and final cost
- **Suggested products carousel** – shows additional items to encourage continued browsing

This structure provides a clear and user-friendly checkout experience while maintaining continuity with the rest of the system.

The basket is managed on the client side using `localStorage` and has a `basket` field in the database for each user, allowing items to persist across page reloads.

Basket data is stored under a shared key:

```
const BASKET_STORAGE_KEY = "ibay_basket";
```

Items are retrieved and safely parsed using:

```
function readBasket() {  
  const raw = localStorage.getItem(BASKET_STORAGE_KEY);  
  return raw ? JSON.parse(raw) : [];  
}
```

This approach ensures that:

- Basket state is persistent
- The interface can be rendered instantly
- Server load is reduced for non-critical operations

When the page loads, the basket is dynamically rendered from localStorage and the Database. Each item is displayed with its title, price, postage, and an option to remove it.

The rendering logic also calculates totals in real time:

```
summaryTotal.textContent = formatGBP(totalPrice + totalPostage);
```

Users can remove items directly from the basket. This is handled on the client-side and server-side using event delegation, ensuring that dynamically created elements remain interactive:

```
listEl.addEventListener("click", (e) => {
  const btn = e.target.closest(".basket-item_remove");
  const next = readBasket().filter((row) => String(row.id) !== String(btn.dataset.basketId));
  writeBasket(next);
  renderBasketFromStorage();
});
```

This provides immediate visual feedback without requiring a page reload or server request.

```
await fetch(`${API_BASE}/purchase_items.php`, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ item_ids: itemIds })
});
```

While the basket is maintained client-side, the checkout process is handled on the server to ensure security and consistency.

When the user clicks “Buy Now”, the client extracts all product IDs and sends them to the backend using a JSON-based AJAX request:

On the server side, the request is validated and processed using PHP. Only valid numeric IDs are accepted, and a prepared statement is used to safely execute the query:

```
$placeholders = implode(',', array_fill(0, count($cleanIds), '?'));
$stmt = mysqli_prepare($conn, $sql);
mysqli_stmt_bind_param($stmt, $types, ...$cleanIds);
```

This ensures:

- Protection against SQL injection
- Data integrity during purchase
- Safe removal or processing of purchased items

After the transaction:

- The client clears the basket (localStorage)
- The basket field in database is cleared
- The user is redirected to the browse page with a confirmation indicator

Strengths

- Clear separation between client-side responsiveness and server-side security
- Efficient use of localStorage to minimise server load
- Dynamic rendering and interaction without page reload
- Secure checkout process using server validation and prepared statements

Limitations

- Basket data stored in localStorage is device specific and not shared across sessions
- Limited validation on client-side basket data

The basket system demonstrates an effective integration of client-side and server-side technologies. By combining fast local interactions with secure backend processing, the system achieves both usability and reliability. This design aligns with modern web development practices, where responsiveness and security must be carefully balanced.

Seller Rating System

```
// Add the new rating to rating_total and increment rating_count
// Then recalculate the mean and store it in the rating column
$query = "UPDATE iBayMembers
SET rating_total = rating_total + $rating,
    rating_count = rating_count + 1
WHERE id = '$sellerId'";
```

Why this design is strong:

- A single LEFT JOIN retrieves both product and seller data in one database query, avoiding multiple trips to the server and keeping the backend efficient.

```
$query = "SELECT
p.productName,
p.price,
p.postage,
p.created_at,
p.description,
p.image_url_1,
p.image_url_2,
p.item_condition,
p.sellerId,
m.email AS seller_email,
m.created_at AS seller_since,
CASE WHEN m.rating_count > 0 THEN m.rating_total / m.rating_count ELSE 0 END AS seller_rating
FROM iBayProducts p
LEFT JOIN iBayMembers m ON p.sellerId = m.id
WHERE p.id = '$productId'";
```

Rate the Seller

How would you rate your experience with this seller?







- Using AJAX to load product data allows the page shell to load instantly, and content is populated dynamically, giving a faster and smoother page.
- The thumbnail swap is handled entirely client-side, keeping simple UI interactions fast and reducing unnecessary server load.
- Mean rating is calculated in SQL rather than PHP, keeping the backend logic efficient.

Known Limitations:

- No restriction preventing a user from rating the same seller multiple times.
- Images rely on filename stored on the database being correct and then the file being stored in the product_images folder. If either is missing the image will not display with no error message shown to the user.

After a product is successfully bought, a popup appears allowing the user to rate the seller out of 5 stars. The popup uses JavaScript hover and click functions to let the user preview and select their rating before submitting.

When submitted, a jQuery AJAX POST request is sent to `submit_rating.php`. The PHP script updates the seller's record in iBayMembers by adding the new score to `rating_total` and incrementing `rating_count`. The mean rating is then calculated directly in the SQL query in `get_product.php`.

Conclusion

The iBay project successfully demonstrates the development of a full-stack client-server web application using modern web technologies. The system provides users with a functional and responsive e-commerce platform where they can register, authenticate, browse products, manage listings, add items to a basket, complete purchases, and interact with seller ratings. Throughout the project, both client-side and server-side technologies were integrated effectively to create a smooth and secure user experience.

The application makes extensive use of HTML, CSS, JavaScript, jQuery, AJAX, PHP, MySQL, and JSON communication. Client-side technologies were used to improve responsiveness through dynamic rendering, asynchronous requests, and real-time validation, while server-side technologies ensured secure authentication, data validation, session management, and reliable database interaction. Security considerations such as password hashing and prepared SQL statements were implemented to reduce vulnerabilities including SQL injection attacks.

A major strength of the system is the clear separation between frontend interactivity and backend processing. Features such as AJAX-driven page updates, dynamic basket rendering, live image previews, and asynchronous product filtering demonstrate the benefits of client-side programming, while secure purchase handling and database operations highlight the importance of server-side processing. The project also demonstrates strong teamwork through the division of responsibilities across authentication, browsing, product management, upload functionality, and basket processing.

Although the system is functional and meets its core objectives, several limitations were identified during development. These include limited password validation, lack of advanced image handling, device-specific basket storage, and restricted seller rating controls. Future improvements could include stronger authentication security, cloud-based basket synchronisation, improved accessibility, image compression, and more advanced recommendation systems.

Overall, the iBay project demonstrates a strong understanding of client-server web development principles and highlights how multiple technologies can be combined to create a secure, responsive, and feature-rich web application. The project provided valuable practical experience in full-stack development, database integration, asynchronous communication, and collaborative software engineering.